

Foundation of Computing

HOME ASSIGNMENT 1

Questions:

1. Define a pointer in C. Explain how pointers provide indirect access to memory using the example:

```
int x = 40;
```

2. Why are pointers called derived data types? Discuss at least three advantages of using pointers in C programs.

3. Explain how pointers help in returning multiple values from a function.
Use the data:

```
a = 18, b = 6
```

Explain what values can be returned using pointers.

4. Explain the address-of (&) operator. Identify four illegal uses of this operator and justify why they are invalid.

5. "All pointer variables occupy the same memory size irrespective of their data type."
Explain this statement and discuss its importance in pointer arithmetic.

6. Explain pointer declaration styles. Why can the following declaration be misleading?

```
int x, *p, y;
```

Rewrite it using best practices.

7. What is a NULL pointer? Explain its significance with respect to program safety and debugging.

8. Explain pointer arithmetic with examples.

Assume:

Base address = 1000

Size of int = 4 bytes

9. Explain the relationship between arrays and pointers.

Why can an array name act like a pointer but cannot be reassigned?

10. Discuss the role of pointers in implementing dynamic data structures such as stacks and linked lists. Why are arrays less flexible in such cases?

11. Write a C program using pointers to calculate the sum and difference of two integers:

a = 25, b = 10

Use a function and return results through pointer arguments.

12.

Write a program using function pointers to perform addition and subtraction on:

x = 12, y = 4

The operation should be selected at runtime.

13. Write a program to store the following strings using an array of character pointers and display them:

India, USA, Japan, France

Q14. Write a program that dynamically allocates memory for:

`n = 5 integers → {5, 10, 15, 20, 25}`

Find the maximum element and free the allocated memory.

15. Write a program where multiple pointers point to the same variable:

`x = 100`

Modify the value using one pointer and display it using another pointer.

16. Write a program to demonstrate pointer reassignment using:

`x = 3, y = 6, z = 9`

Print the value pointed to after each reassignment.

17. Traverse the array:

`int arr[6] = {2, 4, 6, 8, 10, 12};`

using only pointer arithmetic (do not use array indexing).

18. Write a program to reverse the above array in-place using pointers.

19. Given:

`int arr[] = {10, 20, 30, 40, 50};`

Write a program to find the distance between two pointers pointing to `arr[1]` and `arr[4]`.

20. Write a program using a pointer to a two-dimensional array to display all elements of:

`int a[2][3] = {{1, 2, 3}, {4, 5, 6}};`

Explain how pointer dereferencing works in this case.